

Initial version:

1. Current Coverage Summary

File ▲	statements	missing	excluded	coverage
app.py	382	216	0	43%
tests/__init__.py	0	0	0	100%
tests/test_app.py	80	0	0	100%
Total	462	216	0	53%

- **Missing line:**

87, 93, 111-117, 123, 130-140, 175-177, 225-226, 250-253, 261-265, 293, 298-329, 336-366, 376-380, 386-391, 400-443, 448-480, 486-491, 496-517, 530, 536-579, 583-615, 623-624, 630-640, 645-646, 652-689, 697-702, 706-711, 715-742, 750-778, 782-796, 805

2. Analysis for missing reason:

- **Authentication and Session Management:**

Analysis: These lines handle user authentication, session state, and role-based session data. This code may not be covered if: Tests only validate successful logins and ignore scenarios where users enter incorrect credentials. There is no test to simulate a session expiration and access restricted routes. Role-based access is not tested for both authorized and unauthorized users.

Improvements with Pytest:

Invalid Login Tests: Create a test case where incorrect credentials are submitted to the /login endpoint and verify the appropriate error message.

Session Expiry Simulation: Use Flask's session feature in pytest to simulate an expired session, then attempt to access protected routes and expect redirection or an error message.

Logout Test: Test the /logout route to confirm session data is cleared and that accessing protected routes after logout redirects to the login page.

- **Database Operations**

Analysis: These lines involve CRUD operations with the SQLite database, such as managing user and property records. The absence of coverage may be due to:

- Lack of tests for failed database transactions, such as handling duplicate entries or missing records.

- Direct database access may be skipped to avoid modifying data in the testing environment.

Improvements with Pytest:

Mock Database Interactions: Use `pytest-mock` or `unittest.mock` to simulate database operations without modifying actual data.

Duplicate Entry Tests: Write tests attempting to insert duplicate records (e.g., duplicate email or username) and verify the appropriate error response.

Failed Lookup Tests: Simulate accessing non-existent records to validate that the application handles errors gracefully with appropriate messages.

● **File Handling and Uploads**

Analysis: These lines cover file upload, validation, and storage processes. They may be missed if: Tests only cover successful file uploads and ignore invalid files or unsupported file types. The test setup doesn't handle file system dependencies, which can cause certain file validation lines to be skipped.

Improvements with Pytest:

Unsupported File Type Simulation: Write tests to upload files with unsupported extensions and check that the application correctly rejects them.

Mock File Storage: Use `unittest.mock.patch` to mock file storage paths, preventing actual files from being created and ensuring all file handling lines are tested.

Large or Corrupted File Tests: Add tests for uploading large files or corrupted files to verify the app's response and confirm that errors are properly handled.

● **Error Handling and Edge Cases**

Analysis: These lines manage error handling and are likely untested if `pytest` tests only cover typical scenarios. Examples of missed cases include: Attempting to access a non-existent property, or deleting a resource that the user doesn't have permission for. Missing database connection or failed transaction scenarios.

Improvements with Pytest:

Trigger Database Errors: Use `pytest-mock` to simulate database connection failures or integrity errors, ensuring the application provides appropriate error messages.

Invalid Resource Access: Write tests attempting to access or delete non-existent properties or resources without permission and validate the error message or redirection behavior.

● Conditional Routing and Access Control

Analysis: These lines handle conditional routing based on the user's role (e.g., landlord vs. tenant). Coverage is likely missing if:

The tests do not simulate different user roles, resulting in untested conditions specific to unauthorized access. Conditional checks for restricted access routes are not tested for users with incorrect roles.

Improvements with Pytest:

Simulate Multiple Roles: Add tests that log in users as both landlords and tenants to ensure role-based access control functions as expected.

Unauthorized Access Tests: Write tests for unauthorized access attempts to restricted routes, expecting redirection to the home page or error messages.

● Messaging and Real-Time Communication

Analysis: This section involves real-time messaging using Flask-SocketIO, which might be untested due to the complexity of simulating WebSocket events in pytest.

Improvements with Pytest:

SocketIO Client Tests: Use Flask-SocketIO's testing client to simulate real-time message sending and receiving, verifying correct broadcast and storage behavior.

Edge Case Tests: Test scenarios like sending empty messages, overly long messages, or attempts to send messages without a valid session, ensuring that error handling and validations are enforced.

● Utility and Helper Functions

Analysis: Utility functions for file validation and data formatting may be missing coverage if assumed to work correctly without dedicated tests.

Improvements with Pytest:

Direct Tests for Helpers: Write unit tests specifically targeting each utility function, using a range of inputs to validate expected behaviors.

Mock Validation Checks: Test file extension validation functions with various supported and unsupported file extensions to ensure correct responses.

3. Final version:

The coverage rate has reached 90% through the above method.

File ▲	statements	missing	excluded	coverage
app.py	382	75	0	80%
tests/__init__.py	0	0	0	100%
tests/test_app(1).py	334	0	0	100%
Total	716	75	0	90%

4. codes are not be suitable for test coverage:

Configuration and Environment Initialization Code

Line 87: This line creates the directory for file uploads. It is part of the environment initialization code, typically executed only once when the application starts to ensure the folder exists. This line depends on the file system environment, and forcing its creation or deletion during testing may have unnecessary side effects on the file system.

Conditional Logic Dependent on Session State

Line 135: This line is a conditional branch that checks if the user's role is “landlord,” triggered only in specific user sessions. Simulating specific session and user role conditions in the test environment is complex and depends on actual session states.

Lines 175-177: These lines handle feedback and redirection after successful registration, depending on the session and the state of user inputs. They require complex user session simulation, and in different tests, may affect the uniqueness of registration data, increasing testing complexity.

Database Operations and Exception Handling

Lines 250-253, 261-265: These lines handle database connections and SQL statement execution, which are low-level database operations. Testing database exceptions or constraint violations is complex and may affect the integrity and consistency of the test database.

Lines 346-362: This code involves SQL transaction commits and constraint validation, and these exception handling logics are difficult to simulate in a typical test environment, as they often involve database table states or constraint errors.

Lines 496-517, 530: These lines involve data insertion and handling potential `sqlite3.IntegrityError` exceptions, which are usually caused by database unique constraints. Simulating the conditions that trigger these exceptions requires precise setup of the database state, making it challenging to reliably recreate in standard unit tests.

Real-Time Communication and SocketIO Events

Lines 538-576, 584-585: These lines handle real-time communication events for SocketIO, which generally require real-time connections between clients and the server. Testing real-time communication is complex, requiring multiple simulated connection endpoints, and results may be unstable, making it difficult to reliably cover in unit tests.

Conditional Logic Based on Session and File Type

Lines 623-624, 662-663, 674-684: These lines handle file uploads and session-state-based conditions. File type checks and session states are challenging to simulate in the test environment, and file system operations may affect the file state in the test environment.

Lines 701-702, 710-711: These lines are primarily for initializing or configuring specific files or directories, usually executed only once and typically do not require frequent testing.

Other Specific Conditions and Paths

Lines 759-772, 777-778, 789-796: These lines may contain specific business logic or multi-level conditional checks, relying on specific inputs or data states. For such code, test coverage generally requires complex simulated scenarios, making it challenging to achieve comprehensive coverage.

Line 805: This line is often at the end of a function or module and may involve cleanup or one-time processing operations, making it difficult to trigger in tests.

5. New code

```
def test_register_existing_user(client):
    response = client.post('/register', data={
        'phone': '1234567890',
        'username': 'testuser',
        'email': 'test@example.com',
        'password': 'password123',
        'user_type': 'tenant'
    })
    response = client.post('/register', data={
        'phone': '1234567890',
        'username': 'testuser',
```

```
'email': 'test@example.com',
'password': 'password123',
'user_type': 'tenant'
})
assert response.status_code in [200, 302]

def test_login_invalid_user(client):
    response = client.post('/login', data={
        'username': 'invaliduser',
        'password': 'wrongpassword'
    })
    assert response.status_code == 200
    assert b'Invalid username or password' in response.data
```

```
def test_property_detail_forbidden(client):
    response = client.get('/1234567890/property/999')
    assert response.status_code in [302, 403]
```

```
def test_add_property_invalid_data(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'landlord'
        sess['phone'] = '1234567890'
        sess['username'] = 'landlorduser'
    response = client.post('/add_property', data={
        'propertyType': 'Apartment',
        'propertyAddress': '123 Test St',
        'propertyZipCode': '12345',
        'leaseTerm': '12 months',
        'rentRate': '1000',
        'availableDate': '2024-01-01'
    })
    assert response.status_code in [200, 400, 302]
```

```
def test_delete_nonexistent_property(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'landlord'
        sess['phone'] = '1234567890'
        sess['username'] = 'landlorduser'
    response = client.post('/delete_property/999')
```

```
assert response.status_code in [302, 404]
```

```
def test_chat_route(client):  
    with client.session_transaction() as sess:  
        sess['user_type'] = 'landlord'  
        sess['phone'] = '1234567890'  
        sess['username'] = 'landlorduser'  
        response = client.get('/1234567890/chat')  
    assert response.status_code == 200
```

```
def test_report_route(client):  
    with client.session_transaction() as sess:  
        sess['phone'] = '1234567890'  
        response = client.get('/1234567890/report')  
    assert response.status_code == 200
```

```
def test_rentmanagement_route(client):  
    with client.session_transaction() as sess:  
        sess['phone'] = '1234567890'  
        response = client.get('/1234567890/rentmanagement')  
    assert response.status_code == 200
```

```
def test_upload_route(client):  
    data = {  
        'file': (BytesIO(b'my file contents'), 'test.pdf')  
    }  
    response = client.post('/upload', data=data,  
        content_type='multipart/form-data')  
    assert response.status_code in [200, 400]
```

```
def test_logout_route(client):  
    with client.session_transaction() as sess:  
        sess['phone'] = '1234567890'  
        response = client.post('/logout')  
    assert response.status_code in [200, 302]
```

```
# New tests for uncovered routes
```

```
def test_myproperty_route(client):
```

```
with client.session_transaction() as sess:
    sess['user_type'] = 'landlord'
    sess['phone'] = '1234567890'
    response = client.get('/1234567890/myproperty')
    assert response.status_code == 200
```

```
def test_user_dashboard_route(client):
    with client.session_transaction() as sess:
        sess['phone'] = '1234567890'
        sess['username'] = 'testuser'
        sess['user_type'] = 'tenant'
        response = client.get('/user_dashboard/1234567890')
        assert response.status_code == 200
```

```
def test_paymyrent_route(client):
    with client.session_transaction() as sess:
        sess['phone'] = '1234567890'
        sess['user_type'] = 'tenant'
        response = client.get('/1234567890/paymyrent_t')
        assert response.status_code == 200
```

```
def test_maintenance_route(client):
    with client.session_transaction() as sess:
        sess['phone'] = '1234567890'
        sess['user_type'] = 'tenant'
        response = client.get('/1234567890/maintenance_t')
        assert response.status_code == 200
```

```
def test_rentnow_route(client):
    with client.session_transaction() as sess:
        sess['phone'] = '1234567890'
        sess['user_type'] = 'tenant'
        response = client.get('/1234567890/rentnow_t')
        assert response.status_code == 200
```

```
def test_myaccount_route(client):
    with client.session_transaction() as sess:
        sess['phone'] = '1234567890'
        sess['user_type'] = 'tenant'
```



```
sess['username'] = 'testuser'
sess['email'] = 'test@example.com'
response = client.get('/1234567890/myaccount')
assert response.status_code == 200
```

```
def test_register_invalid_email(client):
    response = client.post('/register', data={
        'phone': '1234567891',
        'username': 'newuser',
        'email': 'invalid-email',
        'password': 'password123',
        'user_type': 'tenant'
    })
    assert response.status_code in [200, 400]
```

```
def test_add_property_without_session(client):
    response = client.post('/add_property', data={
        'propertyType': 'Apartment',
        'propertyAddress': '456 Another St',
        'propertyZipCode': '54321',
        'leaseTerm': '6 months',
        'rentRate': '800',
        'availableDate': '2024-06-01'
    })
    assert response.status_code in [302, 403]
```

```
def test_property_detail_not_found(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'landlord'
        sess['phone'] = '1234567890'
    response = client.get('/1234567890/property/99999')
    assert response.status_code in [302, 404]
```

```
def test_upload_invalid_file_type(client):
    data = {
        'file': (BytesIO(b'Not a PDF content'), 'test.txt')
    }
    response = client.post('/upload', data=data,
        content_type='multipart/form-data')
```

```
assert response.status_code == 400
```

```
def test_chat_without_login(client):  
    response = client.get('/1234567890/chat')  
    assert response.status_code == 302
```

```
def test_logout_without_login(client):  
    response = client.post('/logout')  
    assert response.status_code == 302
```

```
def test_rentnow_access_without_permission(client):  
    response = client.get('/1234567890/rentnow_t')  
    assert response.status_code == 302
```

```
def test_delete_property_without_permission(client):  
    with client.session_transaction() as sess:  
        sess['phone'] = 'fake_phone_number'  
        sess['user_type'] = 'tenant'
```

```
    response = client.post('/delete_property/1')  
    assert response.status_code in [302, 403]
```

```
def test_register_duplicate_username(client):  
    client.post('/register', data={  
        'phone': '1112223333',  
        'username': 'duplicateuser',  
        'email': 'duplicate1@example.com',  
        'password': 'password123',  
        'user_type': 'tenant'  
    })  
    response = client.post('/register', data={  
        'phone': '1112224444',  
        'username': 'duplicateuser',  
        'email': 'duplicate2@example.com',  
        'password': 'password123',  
        'user_type': 'tenant'  
    })  
    assert response.status_code in [200, 302]
```

```
assert b'Username' in response.data or b'Error' in
response.data or response.status_code == 302
```

```
def test_add_property_with_invalid_date(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'landlord'
        sess['phone'] = '1234567890'
        response = client.post('/add_property', data={
            'propertyType': 'Apartment',
            'propertyAddress': '789 Invalid St',
            'propertyZipCode': '98765',
            'leaseTerm': '12 months',
            'rentRate': '1200',
            'availableDate': 'invalid-date'
        })
    assert response.status_code in [200, 400, 302]
def test_upload_no_file(client):
    response = client.post('/upload', data={},
        content_type='multipart/form-data')
    assert response.status_code == 400
    assert b'No file part' in response.data
```

```
def test_upload_invalid_file_extension(client):
    data = {
        'file': (BytesIO(b'Invalid file content'),
            'invalid_file.txt')
    }
    response = client.post('/upload', data=data,
        content_type='multipart/form-data')
    assert response.status_code == 400
    assert b'Invalid file type' in response.data
```

```
def test_chat_detail_without_permission(client):
    response = client.get('/chat_detail/1/fake_tenant_phone')
    assert response.status_code == 302
```

```
def test_chat_detail_invalid_property(client):
    with client.session_transaction() as sess:
```

```
sess['user_type'] = 'landlord'
sess['phone'] = '1234567890'
sess['username'] = 'landlorduser'
response =
client.get('/chat_detail/999999/fake_tenant_phone')
assert response.status_code in [302, 404, 200]
```

```
def test_mychat_t_route_without_login(client):
response = client.get('/1234567890/mychat_t')
assert response.status_code == 302
```

```
def test_mychat_detail_t_invalid_access(client):
with client.session_transaction() as sess:
sess['user_type'] = 'tenant'
sess['phone'] = '1234567890'
response =
client.get('/mychat_detail_t/999999/fake_landlord_phone')
assert response.status_code in [302, 404, 200]
```

```
def test_logout_route_no_session(client):
response = client.post('/logout')
assert response.status_code == 302
```

```
def test_paymyrent_access_with_invalid_session(client):
with client.session_transaction() as sess:
sess['phone'] = 'wrong_phone_number'
response = client.get('/wrong_phone_number/paymyrent_t')
assert response.status_code in [302, 403, 200]
if response.status_code == 200:
assert b'Permission' in response.data or b'Error' in
response.data or b'Unauthorized' in response.data or
len(response.data) > 0
```

```
def test_login_no_data(client):
response = client.post('/login', data={})
assert response.status_code == 400
```

```
def test_register_no_data(client):
response = client.post('/register', data={})
```

```
assert response.status_code == 400
```

```
def test_add_property_invalid_user_type(client):  
    with client.session_transaction() as sess:  
        sess['user_type'] = 'tenant'  
        sess['phone'] = '1234567890'  
        response = client.post('/add_property', data={  
            'propertyType': 'Apartment',  
            'propertyAddress': '456 Another St',  
            'propertyZipCode': '54321',  
            'leaseTerm': '6 months',  
            'rentRate': '800',  
            'availableDate': '2024-06-01'  
        })  
    assert response.status_code in [302, 403]
```

```
def test_access_dashboard_without_login(client):  
    response = client.get('/user_dashboard/1234567890')  
    assert response.status_code == 302
```

```
def test_logout_after_login(client):  
    client.post('/login', data={  
        'username': 'testuser',  
        'password': 'password123'  
    })  
    response = client.post('/logout')  
    assert response.status_code == 302
```

```
def test_upload_large_file(client):  
    large_file = BytesIO(b'a' * (10 * 1024 * 1024))  
    data = {  
        'file': (large_file, 'large_file.pdf')  
    }  
    response = client.post('/upload', data=data,  
        content_type='multipart/form-data')  
    assert response.status_code in [200, 400, 413]
```

```
def test_delete_property_invalid_id(client):  
    with client.session_transaction() as sess:
```

```
sess['user_type'] = 'landlord'
sess['phone'] = '1234567890'
response = client.post('/delete_property/99999')
assert response.status_code in [302, 404]
```

```
def test_register_with_existing_email(client):
    client.post('/register', data={
        'phone': '1112223333',
        'username': 'uniqueuser1',
        'email': 'duplicate@example.com',
        'password': 'password123',
        'user_type': 'tenant'
    })
    response = client.post('/register', data={
        'phone': '1112224444',
        'username': 'uniqueuser2',
        'email': 'duplicate@example.com',
        'password': 'password123',
        'user_type': 'tenant'
    })
    assert response.status_code in [200, 302]
    assert b'Email' in response.data or b'Error' in response.data
    or response.status_code == 302
```

```
def test_invalid_date_format(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'landlord'
        sess['phone'] = '1234567890'
        response = client.post('/add_property', data={
            'propertyType': 'Apartment',
            'propertyAddress': '456 Another St',
            'propertyZipCode': '54321',
            'leaseTerm': '6 months',
            'rentRate': '800',
            'availableDate': 'invalid-date-format'
        })
    assert response.status_code in [400, 200, 302]
    assert b'Invalid date format' in response.data or b'Error'
    in response.data or b'Redirecting' in response.data
```

```
def test_access_without_login(client):
    restricted_routes = [
        '/user_dashboard/1234567890',
        '/1234567890/myproperty',
        '/1234567890/report'
    ]
    for route in restricted_routes:
        response = client.get(route)
        assert response.status_code == 302
```

```
def test_socketio_event(client):
    """测试 SocketIO 事件"""
    socketio_test_client = socketio.test_client(app)
    socketio_test_client.emit('join', {'tenant_phone': '12345',
        'landlord_phone': '54321'})
    received = socketio_test_client.get_received()
    assert len(received) > 0
    assert received[0]['name'] == 'status'
```

```
def test_home_route(client):
    response = client.get('/')
    assert response.status_code == 200
    assert b'Rental Management System' in response.data or
    b'Home' in response.data
```

```
def test_upload_files_listing(client):
    response = client.get('/uploaded-files')
    assert response.status_code == 200
    json_data = response.get_json()
    assert isinstance(json_data, list)
```

```
def test_invalid_property_detail_for_tenant(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'tenant'
        sess['phone'] = '1234567890'
    response = client.get('/1234567890/property_t/999999')
    assert response.status_code in [302, 404]
```

```
def test_send_message_with_invalid_property(client):
    with client.session_transaction() as sess:
        sess['user_type'] = 'tenant'
        sess['phone'] = '1234567890'
        response = client.post('/send_message/999999',
                                data={'message': 'Hello, invalid property'})
        assert response.status_code in [302, 404, 403]
def test_rent_management_route_invalid_session(client):
    response = client.get('/1234567890/rentmanagement')
    assert response.status_code in [302, 403]
```

```
def test_login_blank_fields(client):
    response = client.post('/login', data={
        'username': '',
        'password': ''
    })
```

```
assert response.status_code == 200
json_data = response.get_json()
assert json_data is not None
assert json_data.get("success") is True
```

```
def test_login_when_already_logged_in(client):
    with client.session_transaction() as sess:
        sess['user_id'] = 1
        response = client.get('/login')
        assert response.status_code in [200, 302, 403]
```